

Section 1: About You

Goals and Objectives

Why do you want to do a GSoC project with Oppia?	When I came across Oppia, I was immediately drawn to its mission of making education accessible and engaging. I began contributing to Oppia-Android in November 2024, and this journey has been both technically enriching and personally fulfilling. I strongly believe in Oppia's vision and would love to contribute to it meaningfully. What excites me most about Oppia is its innovative approach to learning, incorporating interactive storytelling, engaging visuals, and well-structured lessons to make complex concepts easier to grasp and while understanding its concepts or UX I keep imagining myself as a kid. Beyond the project itself, Oppia has an incredibly welcoming and supportive community. From guidance on PRs to insightful discussions on architecture and best practices in the meetings, the mentorship here has been invaluable in sharpening my Android development and testing skills. Contributing to a GSoC project with Oppia is a unique opportunity to make a lasting impact on open-source education while enhancing my skills, collaborating with a supportive community, and growing as a developer. Additionally, three seniors from my college have previously contributed to the Oppia Foundation. Hearing about their experiences—how they learned clean architecture, scalable code practices, and collaborated with a welcoming and supportive global community—inspired me to follow in their footsteps.
What do you hope to learn/achieve during GSoC?	During GSoC with Oppia, I aim to enhance my Android development skills, particularly in dependency injection (Dagger), and state persistence. I also seek to improve my testing proficiency, including unit testing, UI automation, and ensuring app stability, while strengthening my communication and collaboration skills through code reviews and discussions. Additionally, I want to gain hands-on experience in scalable open-source development and contribute meaningfully to Oppia's mission by building a developer-friendly Platform Parameters Dashboard.
Which Oppia teams have you collaborated with, and what have you done on those teams?	[Android CLaM] Here are some non-coding contributions that I would like to highlight: • Joined almost all CLaM meetings every Tuesday. • Did a thorough investigation and documentation of an existing issue and other related issue of the same type with proper details. [Link] • Helped a newcomer in an approach to an issue and making of a PR. [Link] • Provided clarification on a non-reproducible issue in the developer version of the app, helping to identify inconsistencies between the

		Play Store and locally built versions. This contribution assisted in marking the issue as blocked and guided further troubleshooting. [Link]
Contact information	Name: Ayush Yadav Email: theayushyadav11@gmail.com LinkedIn: https://www.linkedin.com/in/ayush-yadav-7415802a6/ Github: https://www.github.com/theayushyadav11	
Preferred method of communication	Email Google Chat (part of Android CLaM Team)	
Social media handles (optional) <i>This field is optional. We ask for it so that we can tag you in GSoC-related posts if your application is accepted!</i>	LinkedIn Instagram	
Which timezone will you primarily be in during the summer?	Indian Standard Time (+5:30 GMT).	
(If you are a student) When are your school holidays?	I am currently a second-year B.Tech student , and my summer holidays will run from June 2, 2025, to July 21, 2025 , allowing me to dedicate focused time to GSoC during this period.	
What other obligations might you need to work around during the summer?	During the summer, I am fully available and have no other commitments to work around.	
Planned time commitment	My planned time commitment for GSoC will be structured as follows: • During Summer Holidays (June 2 – July 21, 2025): 7–8 hours/day (approximately 30–35 hours/week). • During College (July 22 – September 16, 2025, without exams): 5–6 hours/day (approximately 25–30 hours/week). • During College Exam Period (if applicable): 4 hours/day, ensuring I balance both academics and GSoC effectively. Overall, I anticipate dedicating approximately 370+ hours to the project, aligning with the expected workload. Additionally, I am open to working on weekends if required to meet milestones and ensure timely progress.	

Project Timeframe

Note: Oppia will only be offering a single GSoC coding period timeframe this year, starting on **Jun 2**. All work for Milestone 1 must be completed and submitted by **Jul 4** for internal feedback (with any revisions due by **Jul 18**), and all work for Milestone 2 must be completed and submitted by **Aug 27** for internal feedback (with any revisions due by **Sep 10**). We will not be able to extend these deadlines.

Coding period	I will adhere to the above deadlines.
---------------	---

Communication Channels

Note: The Oppia team places a high emphasis on communication, and we have found that daily contact between contributors and mentors is important for helping keep projects on track. This is why we ask that contributors send short daily updates to their mentors explaining what they have done, where they are stuck, and what they plan to do next.

I can commit to sending daily updates to my mentor by email, each day I work during the GSoC period.	Yes
In addition to the above: how often, and through which channel(s), do you plan on communicating with your mentor?	I plan to maintain regular and clear communication with my mentor throughout the GSoC period: Daily Updates: I will share brief progress updates via email or Google Chat to ensure transparency and keep the workflow smooth. Routine Discussions: For minor queries or clarifications, I will use Google Chat for quick responses. Weekly Meetings: I propose having one mentor meeting per week via Google Meet (or any preferred platform) to discuss progress, receive feedback, and align on upcoming tasks. This structured approach will ensure efficient collaboration, timely feedback, and steady project progress.

Contributor Briefing

Note: A couple of days after the selection results are announced, there will be a mandatory contributor briefing for accepted contributors. Please confirm whether you will be able to attend the briefing.

If accepted, I will be able to attend	Yes
---------------------------------------	---

the contributor briefing at 3 pm UTC on May 10.	
---	--

Section 2: About Your Project

Project Details

Project title <i>(should match the one on the Project Ideas list)</i>	4.2. Platform parameters dashboard
Project size <i>(should match the options on the Project Ideas list)</i>	Medium (~175 hours)
Why did you choose this project?	I chose the Platform Parameters Dashboard project because it aligns perfectly with my interests in Android development, developer tooling, and feature management. While working on my personal projects as well, I often felt the need for a centralized system to manage feature flags and platform parameters. As a developer, I found it frustrating and tedious that enabling one feature for testing could interfere with others, and I wished for a way to toggle features dynamically without modifying code. This project directly addresses that pain point by introducing a dedicated UI for managing feature flags and platform parameters, allowing developers to override settings. By working on this project, I get to contribute to a tool that will empower developers, making Oppia's Android development workflow more efficient while gaining hands-on experience in UI design, state management, and dependency injection.

Required Skills (Project-Specific)

GENERAL	
I can figure out the root cause of an issue, and communicate it effectively using a debugging doc .	Debugging Doc: Pin Input Vanishes on Configuration Change
I can debug and fix CI failures.	Not till now.
I can make changes to	Not till now.

GitHub Actions workflows.	
ANDROID	
Build the app and install it on a local device or emulator. Then verify that you can (a) build a non-test library target locally, (b) run a unit test locally, and (c) play through the app locally.	(a) A Link of a Github gist (b) Screenshot of running and passing StateFragmentTest (c) Video of running and playing through app locally
Write new Kotlin code with unit tests.	- Fixes #4098: Added tests for ComparableOperationSubjectTest - Fixes #4132: Added tests for MathParsingErrorSubject. - Fix #4121: Added tests for TokenSubject - Fixes #4100 : Added tests for PolynomialSubject
Change Android UIs, write tests for them, and manually verify that they work.	- Fixes #3641 Use finish() instead of intent for smoother navigation - Fixes #4294 : Added Profile delete message with AlertDialog

Section 3: Proposal Details

Note: This section is largely adapted from [Oppia TDD Template \(Dec 2024\)](#) – PLEASE MAKE A COPY . We ***strongly recommend reading that doc***, since it has detailed explanations of how to fill out the various sections. It also contains links to sample TDDs, so that you can see what others have done in the past.

Problem Statement

Target Audience	The primary target audience for this project is Developers working on Oppia. These developers need a streamlined way to manage and override platform parameters dynamically without modifying the source code.
Core User Need	Oppia's Android developers need an efficient way to manage and test platform parameters and feature flags without modifying source code and rebuilding the app. Currently, changing feature flags or parameters requires manually editing constants, recompiling the app, and redeploying it, making debugging and testing inefficient. This process slows down development, increases the risk of errors, and makes it difficult to test multiple feature configurations dynamically.

Section 3.1: WHAT

Key User Stories and Tasks that will be implemented

#	Title	User Story Description (role, goal, motivation) <i>"As a ..., I need ..., so that ..."</i>	List of tasks needed to achieve the goal (this is the "User Journey")	Links to mocks / prototypes, and/or PRD sections that spec out additional requirements.
1	Viewing Platform Parameters & Feature Flags in Developer Options	As a developer, I need separate screens for viewing Platform Parameters and Feature Flags so that I can easily check their current values, sync status, and overrides.	The developer navigates to Developer Options from the App Drawer. They see two new options: 1. Platform Parameters 2. Feature Flags.	Developer Options Mocks
			The developer selects either Platform Parameters or Feature Flags, displaying a list of all available parameters/flags. Each entry includes: • Name of the parameter/flag • Current value (which may be from the server, overridden, or default) • Sync status (indicating whether the value is synced from the server, overridden locally, or using the default setting)	Feature Flags Mocks Platform Parameter Mocks
2	Modify Platform Parameters & Feature Flags	As a developer, I need to modify platform parameters and feature flags so that I can test feature configurations dynamically without changing the source code.	The developer selects a parameter or flag to modify. Feature flags have toggle switches to enable/disable features dynamically. Platform parameters have input fields to set custom values (booleans, integers, strings). Changes are stored locally.	Modifying platform Parameter Mocks Modifying Feature Flags Mocks

			A restart dialog appears after modifications, upon navigating back., ensuring changes take effect only after a restart to maintain parameter consistency and prevent unintended behavior. Overridden values persist across restarts.	Restart Dialog Mock
			When a developer overrides a parameter or flag and tries to navigate back then a restart Dialog appears. A "Reset" button becomes available for the respective overridden parameters. Clicking it removes the override entry from the local storage. The parameter value is then resolved in the following order: 1. If a remote server value is available, it takes precedence and is applied. 2. If no remote value exists, the parameter reverts to its local default value.	Reset button Mock
3	Force Download Latest Flags & Parameters	As a developer, I need a way to manually fetch the latest platform parameters and feature flags from the remote server so that I can ensure I'm working with up-to-date values.	Add a "Force Download Remote Parameters" option to refresh remote values. A restart should be required to apply the newly downloaded values. A restart prompt should appear, informing the developer that the app needs to restart to apply changes. After restart, the newly downloaded values should be loaded..	Force Downloading Mocks
4	Survive	As a developer, I	Ensure overridden values and UI state	

	Configurati on Changes	need the UI state to persist across configuration changes (e.g., screen rotation) so that I don't lose my modifications.	persist after configuration changes. Test behavior across device rotations.	
--	---------------------------	--	--	--

Testing Plan

Critical user journeys

User Type	User Story Description	Test definition		
		Setup (ends with a user of the correct type getting logged in)	Test steps	Test expectations
Developer	View all feature flags in the Feature Flags screen.	1. Ensure the app is running in the DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options .	1. Open Developer Options . 2. Click on Feature Flags .	The Feature Flags screen correctly displays all feature flags.
			3. Verify that all feature flags are displayed with their names, toggle states (ON/OFF), and sync status .	Each flag shows the correct name, state (ON/OFF), and sync status .
			4. Cross-check displayed default values with feature_flags.textproto for correctness.	No missing or mismatched flags.
Developer	View all platform parameters in the Platform Parameters screen.	1. Ensure the app is running in the DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options .	1. Open Developer Options . 2. Click on Platform Parameters .	The Platform Parameters screen correctly displays all parameters.
			3. Verify that all parameters are displayed with their names, current values, and sync status (server/default/overridden) .	Each parameter shows the correct name, value, and sync status .

			4. Cross-check displayed values with platform_parameters.textproto for correctness.	No missing or mismatched parameters.
Developer	Modify a Feature Flag and verify changes persist.	1. Ensure the app is running in the Developer Flavor . 2. Launch the app. 3. Navigate to Developer Options → Feature Flags ..	1. Open Developer Options . 2. Click on Feature Flags .	The Feature Flags screen correctly displays all feature flags.
			3. Locate a specific Feature Flag (e.g., <i>enable_multiple_classrooms</i>). 4. Toggle the flag ON .	The toggle switch updates instantly when changed. A restart is required for changes to take effect. The modified state persists after restart.
			5. Attempt to navigate back → A restart dialog should appear. 6. Click " Restart " to apply the change.	A restart is required for changes to take effect. On clicking Restart App restarts.
			7. After restart, return to Feature Flags and verify the flag is still ON and correctly applied.	The modified state persists after restart. The feature behaves as expected:
			8. For example : Check the expected app behavior (e.g., if <i>enable_multiple_classrooms</i> is ON, multiple classroom carousels should be displayed).	The feature behaves as expected: → If <i>enable_multiple_classrooms</i> is ON , multiple classrooms are displayed .
			9. Toggle the flag OFF and repeat the restart process. 8. After restart, verify the flag is OFF and confirm that multiple classroom carousels are no longer displayed .	→ If <i>enable_multiple_classrooms</i> is OFF , multiple classrooms are hidden .

Developer	Reset values to defaults. Verify that clicking Reset restores default values.	1. Ensure the app is running in the DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options .	1. Open Developer Options . 2. Click on Platform Parameters .	The Platform Parameters screen correctly displays all parameters.
			3. Modify a platform parameter.	The value changes in the input box or switch is toggled.
			4. Try navigating back.	The Restart dialog appears.
			5. Click Restart.	The app restarts.
			6. Enter the platform parameters screen again. 7. Click the "Reset" button below the modified parameter.	The value updates back to default/ server value. The reset button disables.
			8. Try navigating back.	The Restart dialog appears.
			9. Click Restart.	The app restarts.
Developer	Ensure different parameter types accept only valid inputs .	1. Ensure the app is running in the Developer Flavor . 2. Launch the app. 3. Navigate to Developer Options → Platform Parameters .	1. Open Developer Options . 2. Click on Platform Parameters .	The Platform Parameters screen correctly displays all parameters.
			3. Enter an invalid value for a numerical parameter (e.g., text instead of a number)	The app prevents invalid input . An error message appears, the text box outline turns red, and a message says "Invalid Value" .
			4. Enter an out-of-range value (if applicable).	An appropriate error message appears indicating the value is out of range
			5. Attempt to save changes .	Changes are not saved. Only valid changes are accepted and persisted.
Developer	Ensure the Reset button appears only for overridden values.	1. Ensure the app is running in the DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options .	1. Open Developer Options . 2. Click on Platform Parameters/Feature Flags .	The Platform Parameters/Feature Flags screen correctly displays all parameters/flags.
			3. Check if Reset buttons are visible before making any changes.	Reset buttons should not be visible initially for parameters/flags that are using default/server values.

			4. Modify a feature flag or parameter (e.g., change a value or toggle a flag).	The selected flag/parameter reflects the new overridden value.
			5. Try navigating back and restarting the app.	The app gets restarted.
			6. Open the platform Parameter screen and verify that the Reset button appears next to the modified parameter/flag.	The Reset button becomes visible, indicating that the value has been overridden.
Developer	Ensure that no restart prompt appears when no values are modified.	1. Ensure the app is running in the DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options .	1. Open Developer Options . 2. Click on Platform Parameters .	The Platform Parameters screen correctly displays all parameters.
			3. Navigate through the screen without making any changes . 4. Attempt to navigate back.	No restart prompt should appear if nothing was modified.
Developer	Ensure modified values survive configuration changes.	1. Run the app in DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options → Platform Parameters / Feature Flags .	1. Modify a Feature Flag / Platform Parameter . E.g. Change the switch of <i>DOWNLOADS_SUPPORT</i> from disable to enable.	That particular switch next to <i>DOWNLOADS_SUPPORT</i> enables.
			2. Rotate the device.	The screen is rotated.
			3. Verify that the configuration change does not change any value.	The switch of <i>DOWNLOADS_SUPPORT</i> is still enabled. No unexpected resets or value losses.

Developer	Verify the correct display of sync status badges (Server, Overridden, Default).	1. Run the app in DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options → Platform Parameters / Feature Flags .	1. Verify that parameters/flags synced from the server display a "Server" badge.	"Server" badge appears when values are synced from the server. "Default" badge appears when no remote or overridden value exists. After reset, the badge correctly reflects the source of the value.
			2. Modify a parameter/flag and verify that it now displays an "Overridden" badge.	"Overridden" badge appears when a parameter/flag is modified.
			3. Click the "Reset" button and navigate back, a restart dialog appears. 4. Click on the restart button and then come back.	Badge changes back to "Server" or "Default"
Developer	Ensure that the force download option updates values correctly and handles configuration changes.	1. Run the app in DEVELOPER flavor (<i>BuildFlavor.DEVELOPER</i>). 2. Launch the app. 3. Navigate to Developer Options	1. Click the "Force Download Remote Parameters" button.	Force Download button is available and functions correctly as on clicking on it download starts. An alert Dialog appears with the title "Remote Platform Parameters" and with the description "Downloading..."
			2. Wait for the download to finish.	After the download finishes the description appears as "Successfully Downloaded" and the "Restart" button appears.
			3. While waiting for download the dialog has a "Cancel" button to cancel the downloading. Click the cancel button.	Downloading stops and the Alert Dialog disappears.
			4. After a successful	After restart, downloaded

			download followed by restart, return to verify updated values in the dashboard and new parameter values reflected in app behavior.	values are correctly applied.
--	--	--	--	--------------------------------------

Technical Requirements

UI Screens/Components

#	ID	Description of new UI component	i18n required?	Mock/spec links	What A11y support will be implemented?
1.	feature_flags_menu_item	A new option labeled " Feature Flags " is added to the Developer Options screen. Selecting it navigates to the Feature Flags screen.	No – Developer options menus are not localized per guidelines.	Developer Options Menu Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact with the interface seamlessly, with their personalized accessibility settings .
2.	platform_parameters_menu_item	A new option labeled " Platform Parameters " is added to the Developer Options screen. Selecting it navigates to the Platform Parameters screen.	No – Developer options menus are not localized per guidelines.	Developer Options Menu Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact with the interface seamlessly, with their personalized accessibility settings .
3.	feature_flags_screen	A new screen listing Feature Flags with: • Toggle switch for enabling/disabling each flag dynamically. • Sync status indicator	No	Feature Flags Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact

		showing whether the flag value is from the Override or Server or Default. • Reset button for overridden flags, allowing developers to revert them to production server / default fallback values.			with the interface seamlessly, with their personalized accessibility settings .
4.	platform_parameters_screen	A new screen listing Platform Parameters with: • Editable field (Number input for numbers and text input for Strings, toggles for boolean values). • Sync status indicator showing whether the flag value is from the Override or Server or Default. • Reset button for overridden flags, allowing developers to revert them to production server / default fallback values.	No	Platform parameter Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact with the interface seamlessly, with their personalized accessibility settings .
5.	restart_confirmation_dialog	A confirmation dialog appears when a developer tries to navigate back after modifying any platform parameter, prompting a “Restart” to apply changes. Ensures changes take effect only after a restart. Blocks further edits until dismissed.	No	Restart Alert Dialog Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact with the interface seamlessly, with their personalized accessibility settings .
6.	force_download_button	A “Force Download” button is added as a new menu item in the Developer Options screen, allowing developers to fetch the latest values from the server. → Platform Parameters require a restart for changes to take effect. → A restart prompt appears after downloading parameters.	No	Force download Mocks	When TalkBack is enabled, users will be able to navigate, read, and interact with the interface seamlessly, with their personalized accessibility settings .

Data Handling and Privacy

#	Type of data	Description	Why do we need to store this data?	Anonymized?	Can the user opt out?	Wipeout policy	Takeout policy
1.	Platform Parameter Overrides	Stores user-modified values for platform parameters, replacing default or server-synced values.	Required to persist overridden platform parameters across app restarts and ensure accurate testing conditions.	No, as Platform parameters are specific to the developer's environment.	Yes, by resetting parameters to default within the UI.	Deleted when the app is uninstalled or parameters are reset manually.	Not applicable.
2.	Feature Flag Overrides	Stores the developer's enabled/disabled state for feature flags.	Required to allow developers to toggle features for testing without modifying source code.	No, as feature flags are specific to the developer's environment.	Yes, by resetting the feature flags to default.	Deleted when the app is uninstalled or when feature flags are reset manually.	Not applicable.

Connection to Existing Work

This feature is directly built upon the foundational work introduced in [PR #5725](#), which laid the groundwork for the platform parameter system by introducing the core data models and syncing infrastructure for fetching and applying platform parameters and feature flags from the backend.

Additionally, this proposal builds on the existing **Developer Options Menu**, which already supports routing to experimental development tools and system controls used by developers and testers. The integration of platform parameters and feature flags into this menu aligns with prior design decisions around centralized developer utilities, ensuring consistent UX for debugging and feature toggling.

Together, these existing systems provide the underlying mechanisms—both backend-connected and locally-configurable—that this project will enhance through a UI-based management interface and override functionality.

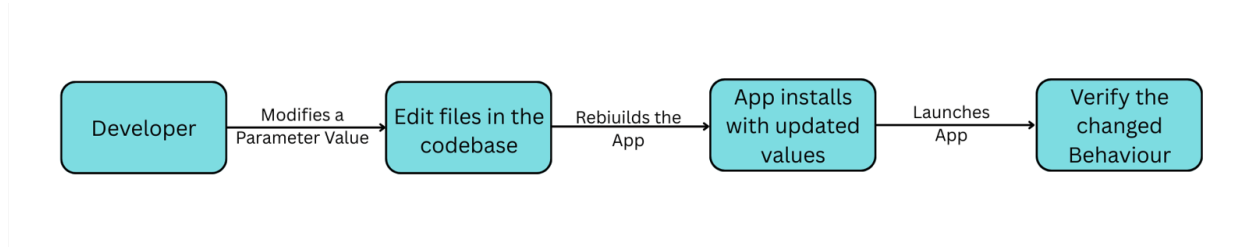
Section 3.2: HOW

Existing Status Quo

1. Absence of In-App Platform Parameter Controls

Currently, **there is no dedicated UI dashboard** in the Oppia Android app to manage **Platform Parameters** or **Feature Flags** through the Developer Options menu. Developers have to manually make changes in the code and re-build the app if they want to change/modify any feature flags or Platform Parameters.

The current flow can be described by the following diagram:



The developer options in our application currently lack crucial functionality for managing platform parameters. Developers face significant limitations because:

- Platform parameter values cannot be viewed or modified directly within the application, requiring source code changes and a full rebuild for any parameter adjustments.
- There is no visibility into parameter states (default, remote, or locally overridden) while testing, making debugging configuration-related issues time-consuming and inefficient.
- Local parameter overrides for testing purposes require modifying code files rather than using developer-friendly interfaces, increasing development friction.

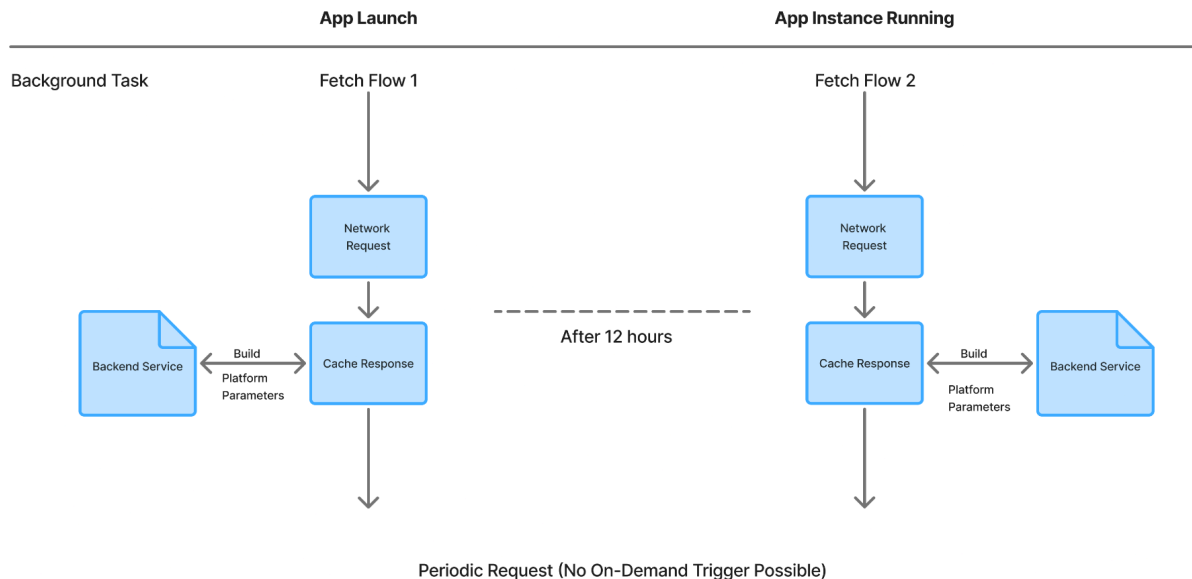
These limitations significantly slow the development cycle, especially when fine-tuning feature flags or configuration parameters that require repeated adjustments during testing phases

2. No Support for On-Demand Download of Remote Platform Parameters

The current version of the app has the **inability to force download remote platform parameter values from the server** at any required time.

- We cannot manually download the remote values of the flags and parameters from the Oppia Web server.
- Currently, remote platform parameters are scheduled for download using **WorkManager** which triggers a network request periodically to run in background with a predefined time gap to fetch the latest Parameter values from the backend.
- The periodic nature of the current approach, does not provide the flexibility to download the parameters at any required time, limits its effectiveness for on-demand updates

The following diagram explains the current system to download the remote parameters:



[Credit: [GSoC 2021 : Platform Parameter Implementation Proposal - Arjun Gupta](#)]

As from the [wiki](#) Feature Flags allows the team to stage features behind remotely configurable flags until they're ready to be launched. But developers have no direct way within the app to view or modify these values dynamically. The introduction of a **Platform Parameters & Feature Flags dashboard** aims to solve this problem. Similarly for the **Force Downloading** where we used to fetch from the server using Work Manager but cannot manually download them.

Solution Overview

Dagger Gating with Dev and Prod Flavors

Here's a **detailed explanation** of **how developer-only UIs (like Developer Options) are gated and isolated** in Oppia Android.

Oppia uses **build flavors** to define distinct app variants in [build flavors.bzl](#):

```
AVAILABLE_FLAVORS = [
    "dev",
    "alpha",
    "beta",
    "ga",
]
```

Selection of the Application Classes:

Oppia Android uses separate Application classes for each Build flavor which are as follows:

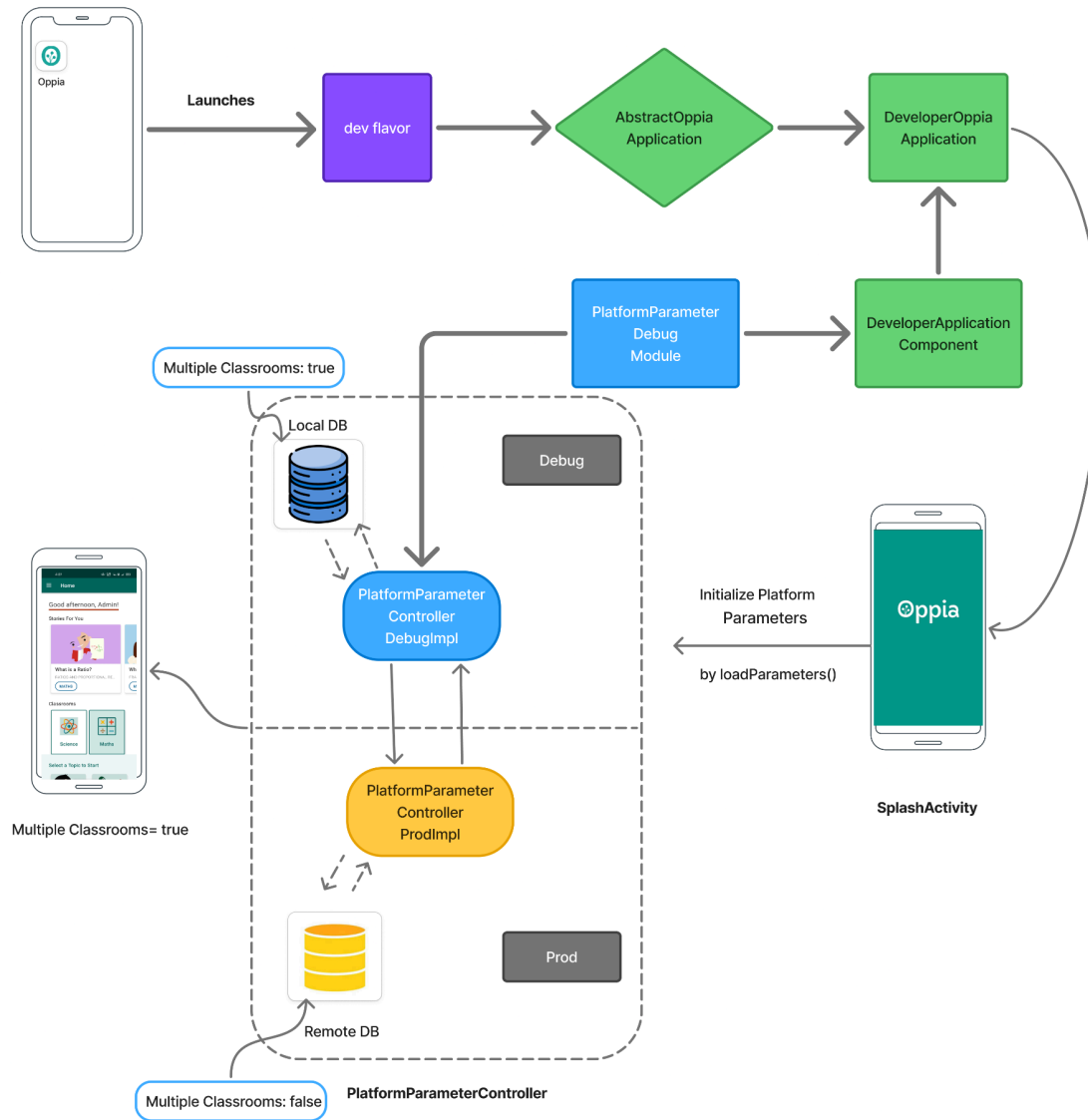
1. dev flavor -> DeveloperOppiaApplication
2. beta flavor -> BetaOppiaApplication
3. alpha flavor -> AlphaOppiaApplication
4. ga flavor -> GaOppiaApplication

Each **Dagger Component** determines **which modules are included** at compile-time.

Modules like *DeveloperOptionsModule*, and any future debug modules (e.g., *PlatformParameterDebugModule*) are included **only** in the [*DeveloperApplicationComponent*](#) .

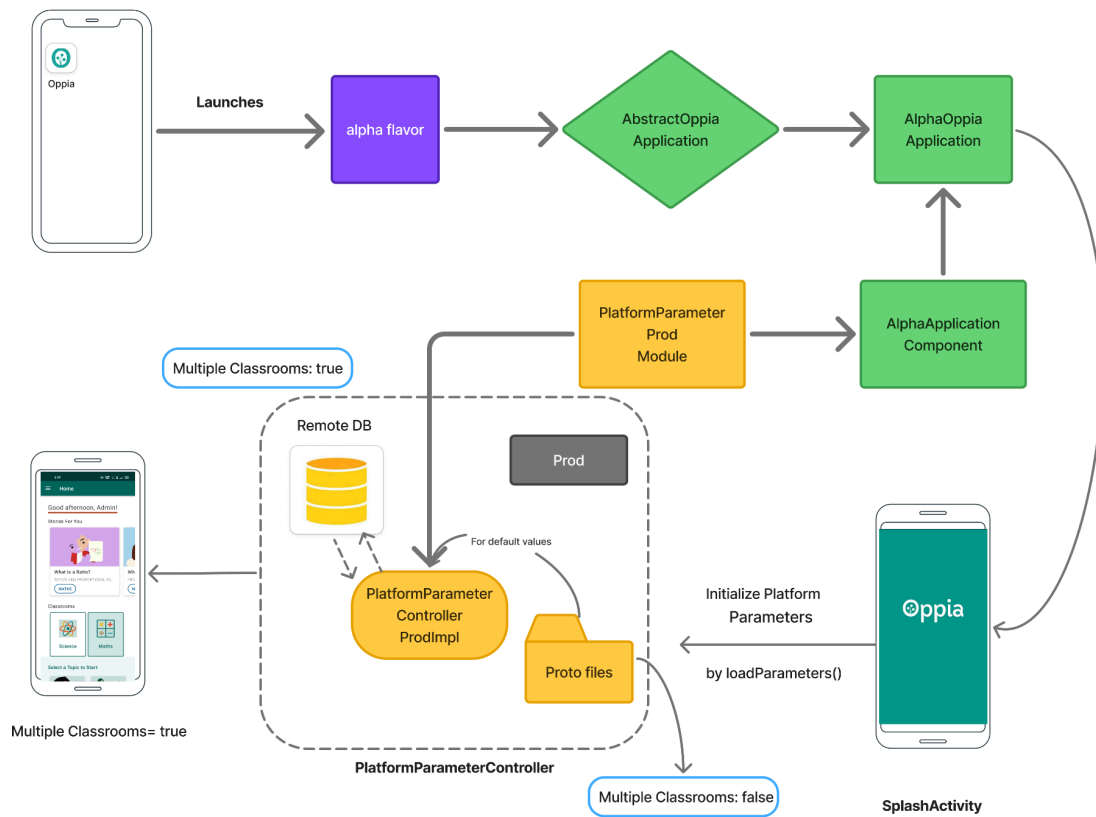
The remote Server is accessed only through the ProdImp but is implemented by the DebugImp relying on the ProdImp.

The following diagram explains the dev flavor of the app and modules are gated:



For the production flavor as we have three flavors namely alpha, beta and ga. So only the prod module will be gated into the application when the app is running in these three flavors. The debug module is not included in these flavors.

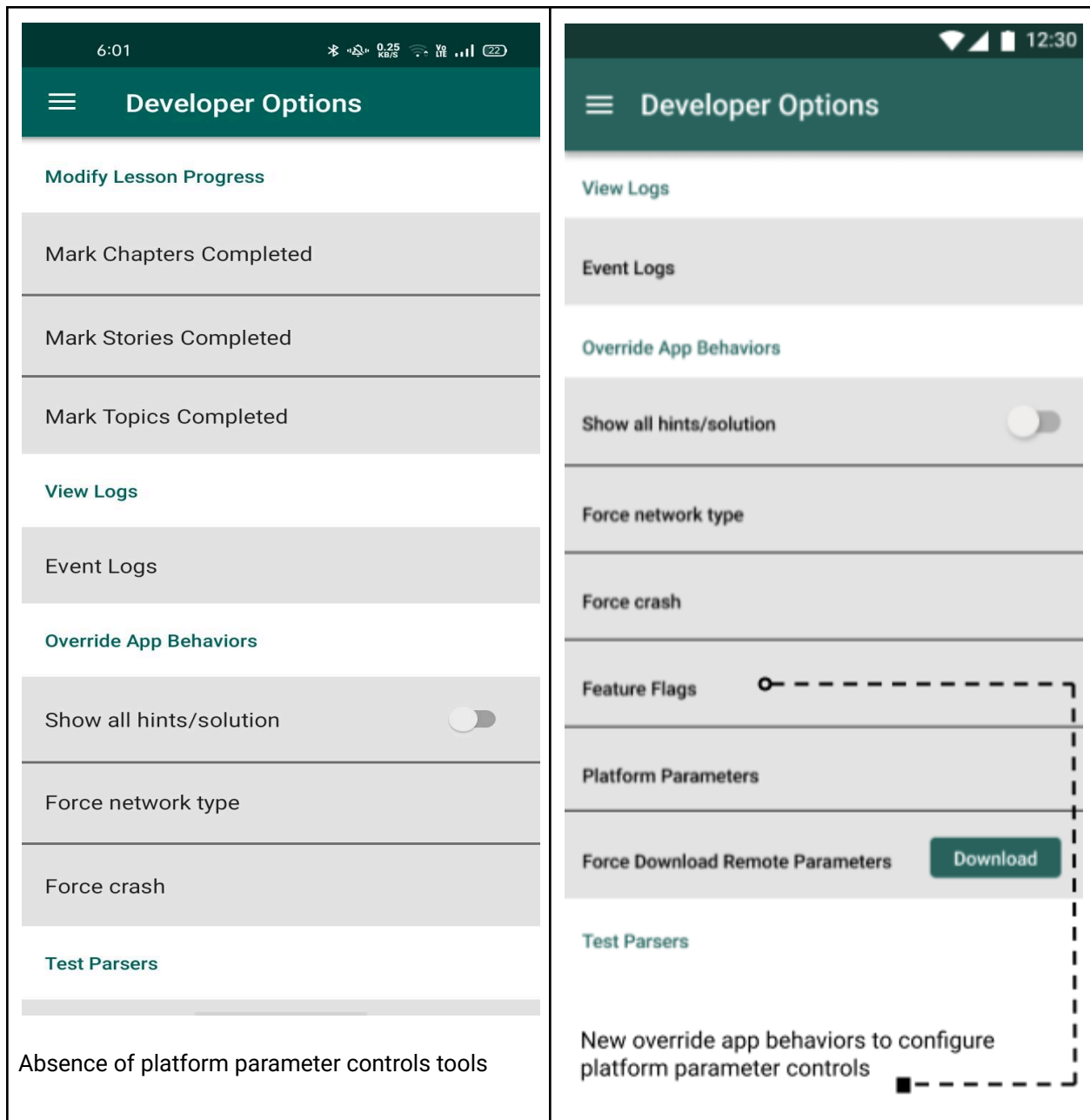
The following diagram explains the production(e.g. alpha) flavor of the app and modules are gated:



1. Platform Parameter Dashboard

The proposed solution introduces a dedicated dashboard within the Developer Options of the Oppia Android app to manage platform parameters and feature flags dynamically at runtime.

Existing Dashboard	Proposed New Menu Dashboard Items
--------------------	-----------------------------------



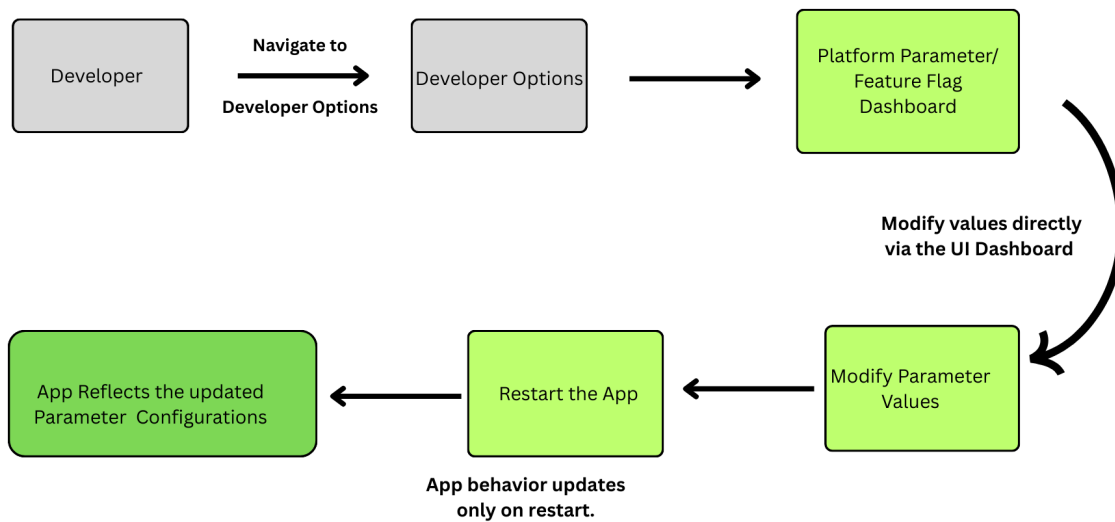
This dashboard will allow developers to:

- **View** all platform parameters and feature flags along with their current values and sync statuses.
- **Modify** these values via intuitive UI elements (toggle switches for feature flags, editable input fields for platform parameters).
- **Persist** overridden values using a new persistent cache store.
- **Reset** changes to revert to either the server-synced value (if available) or the local default.
- **Force Download** the latest remote values on demand.

By doing so, the solution addresses the limitations of the current system, where parameters and flags are statically defined in code (via protos) and require a rebuild to modify, thereby enhancing testing efficiency and developer productivity.

Solution Flow

This reduces the tedious job to manually change the code and rebuild the app again and again. The flow after the completion of the Dashboard will be: Now



This solution now enables the developer to manually override the feature flag or platform parameters from the dashboard itself making it a more efficient tool for testing.

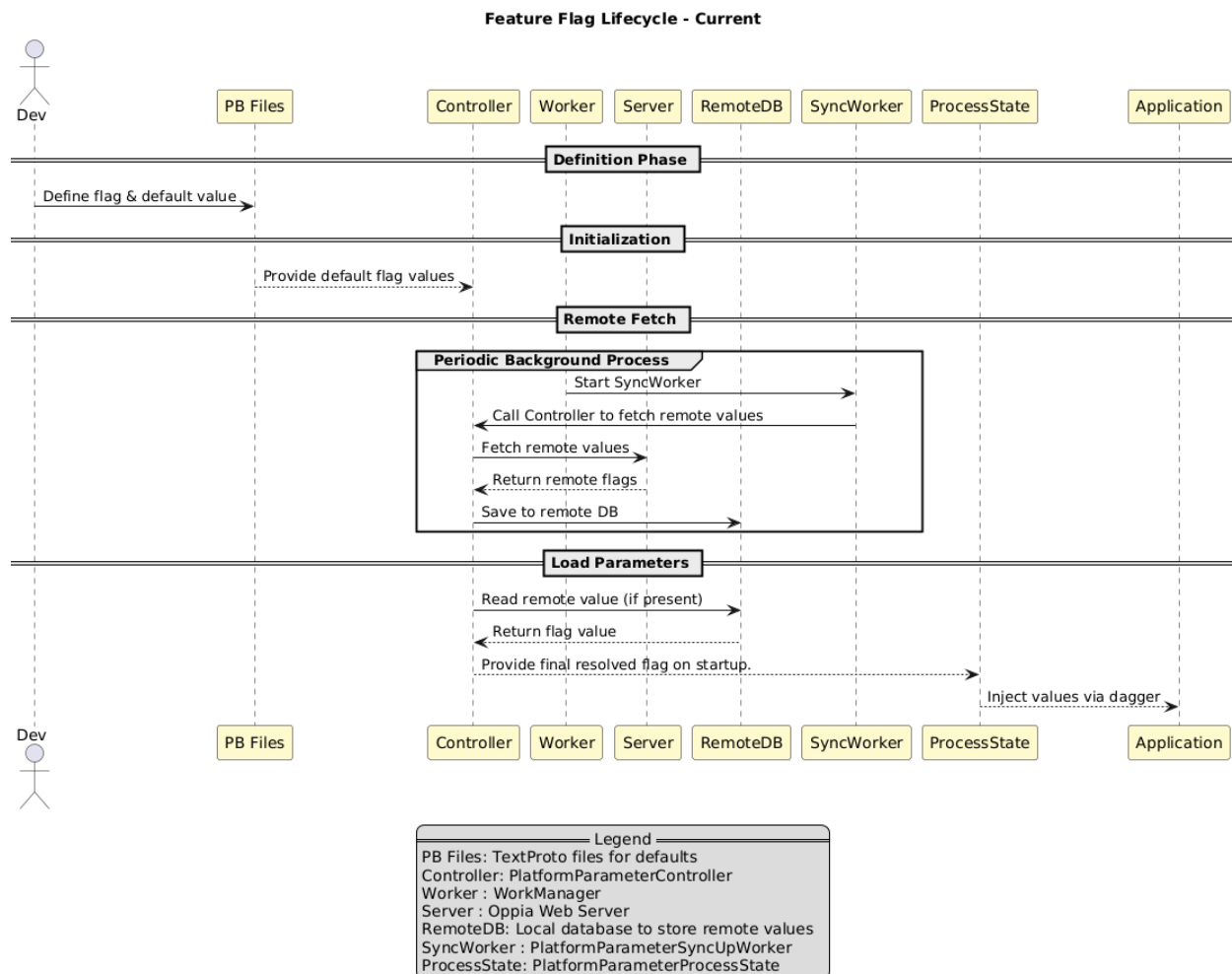
Working of a feature Flag:

Platform Parameter / Feature Flag Lifecycle

In the current implementation of **Platform Parameters (Feature Flags)** in Oppia Android:

- **Default values** for all parameters are defined statically in a `platform_parameter_database.proto` file.
- These values are injected into the application components at compile-time via Dagger.
- At runtime, a **WorkManager** task is triggered (typically on app startup or periodically) to fetch **updated remote parameter values** from the backend server.
- When the app components need a parameter (e.g., `show_multiple_classrooms`), they consult this database via `PlatformParameterProdImpl`.

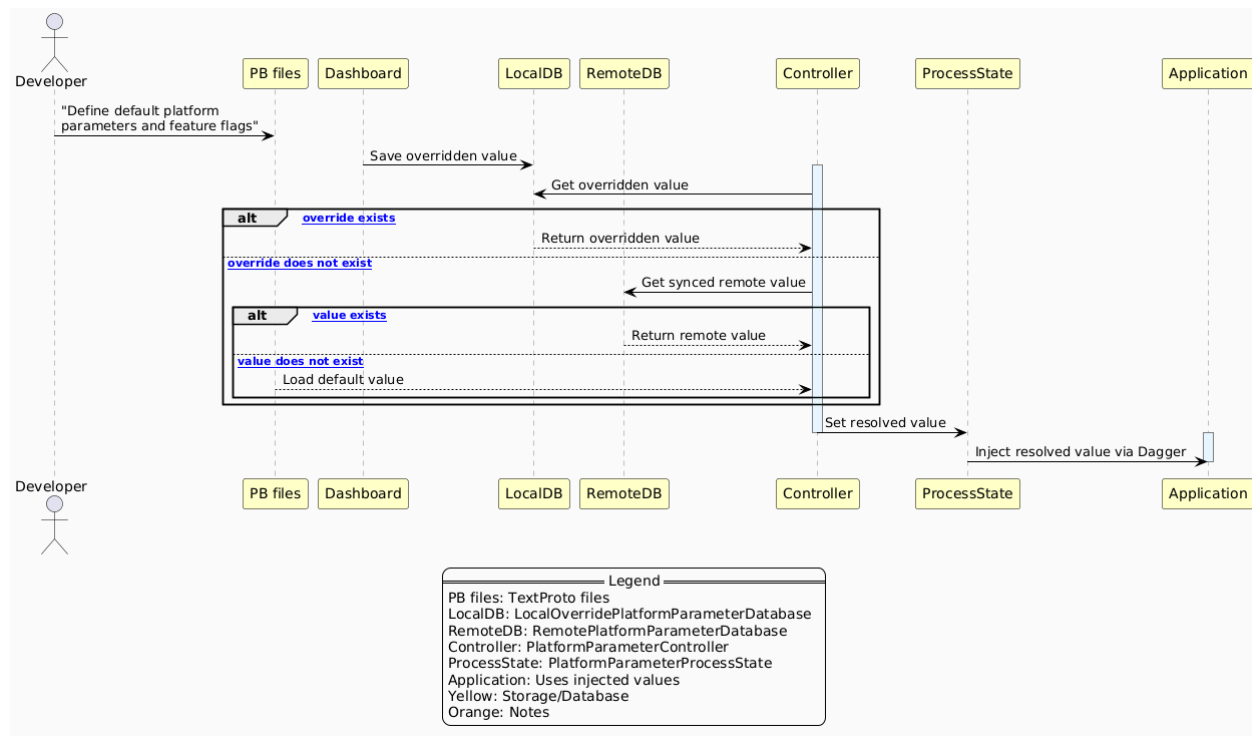
Diagram:



This project addresses the limitations outlined in the status quo by introducing a dedicated dashboard that allows developers to conveniently view and modify platform parameters and feature flags within the app. Alongside the existing default (proto-defined) and remote (server-synced) values, the system now supports a **local override value**, enabling temporary value changes without server-side modifications.

Furthermore, a new manual **"Download Remote Parameters"** button is integrated into the dashboard, allowing developers to trigger an on-demand sync of the latest server values. These values are written to local storage but only take effect upon app restart, ensuring stability and preventing mid-session inconsistencies. Overall, this solution improves visibility, control, and flexibility for managing feature flags and platform parameters in a developer-friendly manner. Overridden values are stored using the **PersistentCacheStore** and are designed to persist across app restarts and configuration changes.

Diagram:



To enable in-app management of Platform Parameters and Feature Flags, the following major changes will be introduced:

1. Override Storage System

A new proto-based *LocalOverridePlatformParameterDatabase* will be introduced. This will store developer-specific overrides separately from the synced remote values. It enables:

- Persistent storage of overridden values across app restarts.
- Easy reset of overrides back to their synced or default values.

To support this, a new *LOCAL_OVERRIDE* enum value will be added to the platform parameter sync status. The UI can use this to display override indicators (e.g., badges or chips).

2. Debug Controller for Developer Builds

Files to be added:

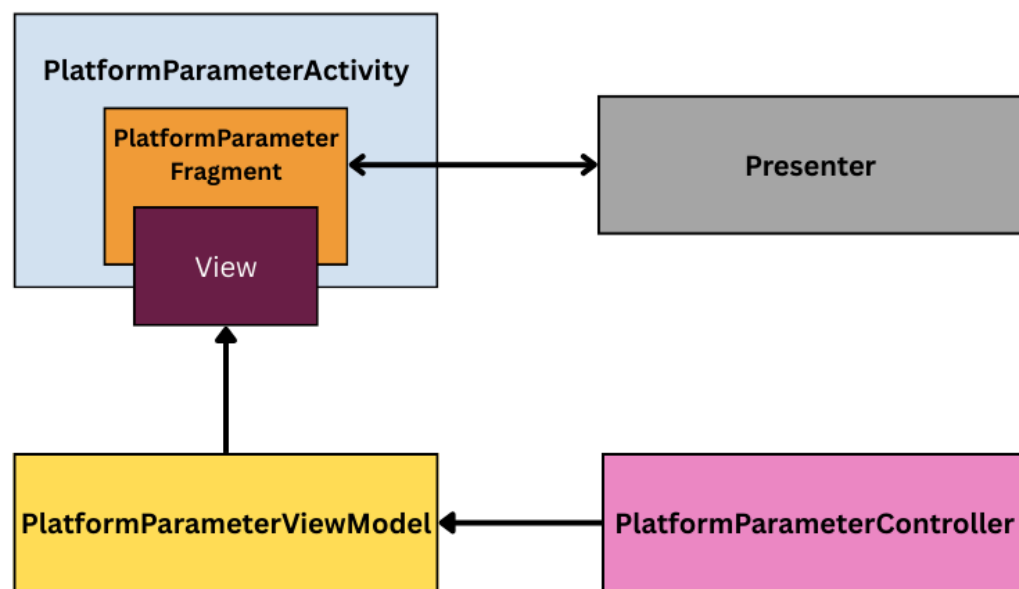
- domain/platformparameter/PlatformParameterControllerDebugImpl

A specialized debug-only implementation of the controller, *PlatformParameterControllerDebugImpl*, will wrap the production implementation. Its responsibility includes:

- Merging locally overridden values on top of the existing remote/default values.
 - Loading parameters at app startup to ensure immediate effect.
 - Supporting the override workflow from the Developer Options screen.
- This debug controller will be conditionally provided via Dagger only in the developer build flavor of the app.

3. UI Support for Platform Parameters and Feature Flags and Force Download

The subsequent sequence diagram illustrates the interaction between PlatformParameterActivity, PlatformParameterFragment, PlatformParameterViewModel and PlatformParameterController.



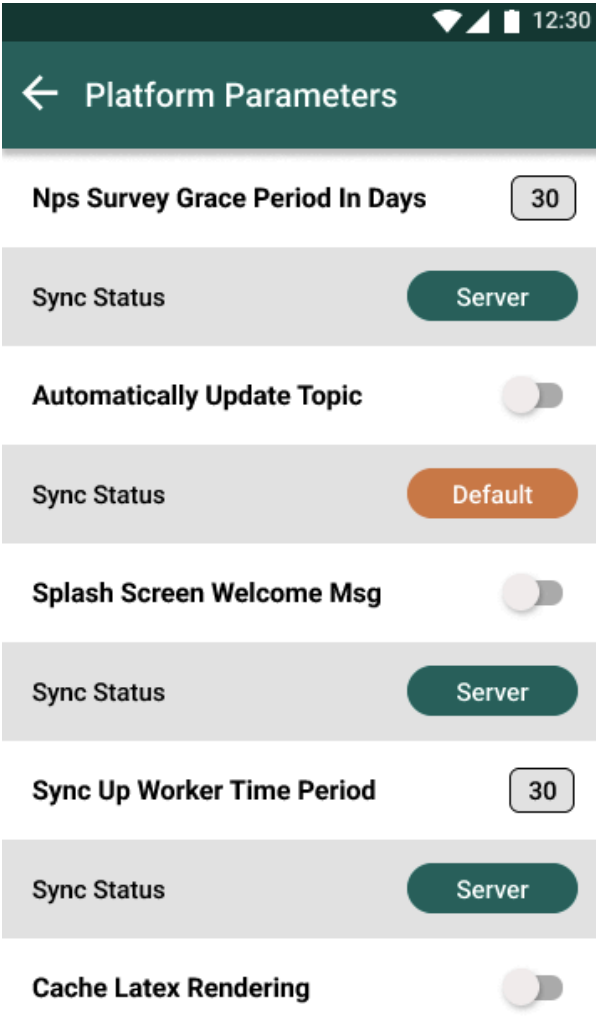
[Credit: Diagram inspired from- <https://github.com/oppia/oppia/wiki/pdfs/GSoC2024SaptakManna.pdf>]

Platform Parameters Screen

Files to be added:

- app/devoptions/platformparameter/PlatformParameterActivity.kt
- app/devoptions/platformparameter/PlatformParameterActivityPresenter.kt
- app/devoptions/platformparameter/PlatformParameterFragment.kt
- app/devoptions/platformparameter/PlatformParameterFragmentPresenter.kt

- app/devoptions/platformparameter/PlatformParameterViewModel.kt
- res/layout/platform_parameter_activity.xml
- res/layout/platform_parameter_fragment.xml
- res/layout/platform_parameter_item_view.xml
- app/devoptions/RouteToPlatformParametersListener.kt

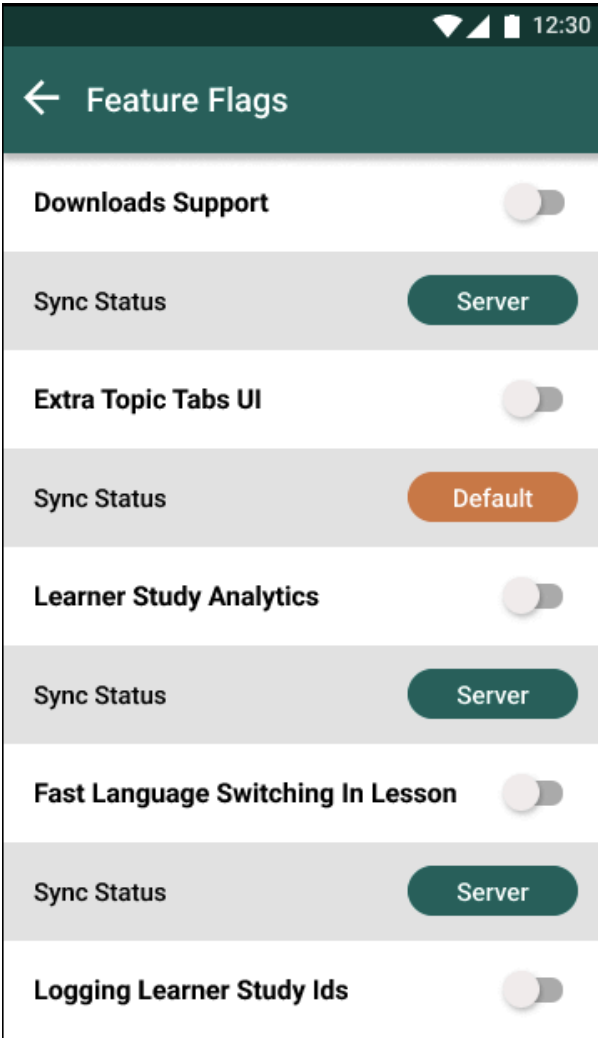
UI Mock-Up	Solution Overview
	• Will show a list of all registered platform parameters. • Displays input fields to represent the current value. • Displays the source of value (default, remote, or override) • Indicates the source of the currently applied value using status chips. • Includes a restart confirmation dialog when changes are made.

Feature Flags Screen

Files to be added:

- app/devoptions/featureflag/FeatureFlagActivity.kt

- app/devoptions/featureflag/FeatureFlagActivityPresenter.kt
- app/devoptions/featureflag/FeatureFlagFragment.kt
- app/devoptions/featureflag/FeatureFlagFragmentPresenter.kt
- app/devoptions/featureflag/FeatureFlagViewModel.kt
- res/layout/feature_flag_activity.xml
- res/layout/feature_flag_fragment.xml
- res/layout/feature_flag_item_view.xml
- app/devoptions/RouteToFeatureFlagsListener.kt

UI Mock-Up	Solution Overview
	• Will show a list of all registered feature flags. • Displays a toggle representing enabled or disabled state. • Displays the source of value (default, remote, or override) • Indicates the source of the currently applied value using status chips. • Includes a restart confirmation dialog when changes are made.

Similar in design to the parameters screen but focused on *Boolean* feature flags and supports toggling features on/off, resetting, and understanding current sync source.

4. Test Infrastructure

Test activities will be introduced to allow isolated testing of these fragments:

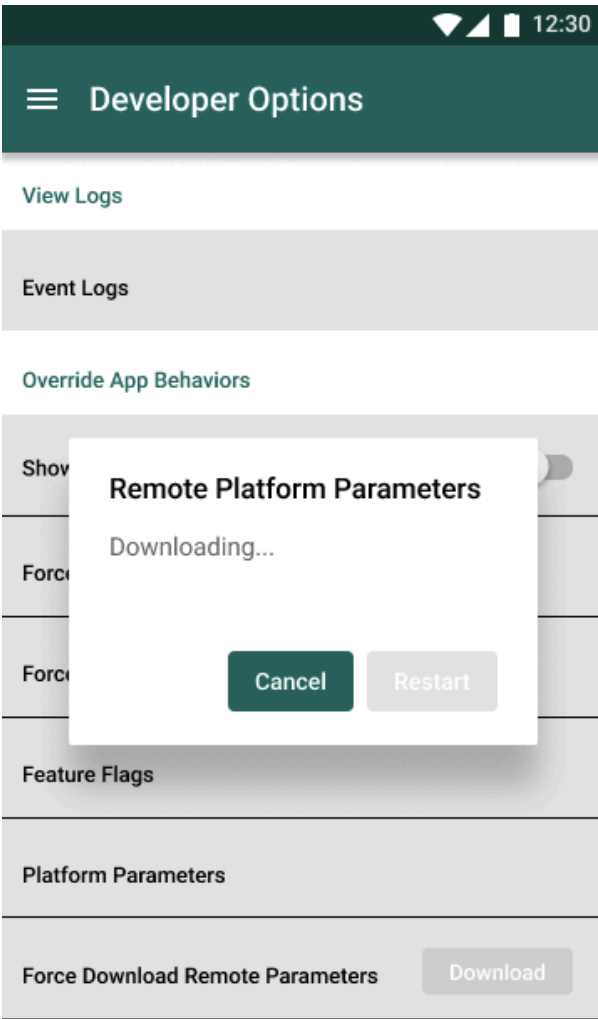
- Separate test activities for both screens (in *testing/* folders).
- Corresponding test fragments in *sharedTest* for behavior verification.
- UI tests will ensure value changes, persistence, sync source display, and restart dialogs function correctly.
- We will add the tests for the debug implementation of the controller:
"PlatformParameterControllerDebugImplTest".

2. Force Remote Download

An additional "Force Download Remote Parameters" button may be introduced to:

- Trigger a manual fetch of the latest remote platform parameters and feature flag values.
- Queue the updated values for use **after an app restart**.

UI Mock-Up	Solution Overview
------------	-------------------



- The force downloading menu item will contain a *TextView* with the text “Force Remote Download Parameter” and a *Button* with the text “Download”.
- On download it prompts an alert dialog with “Restart” (disabled) and “Cancel” options.
- On downloading it displays the status as “Downloading”
- Once download is completed it updates with a status “Successfully Downloaded”
- And alerts the user to Restart the app to apply the updated values.
- Optionally if stuck on downloading user can opt to cancel the download..

Files to be added:

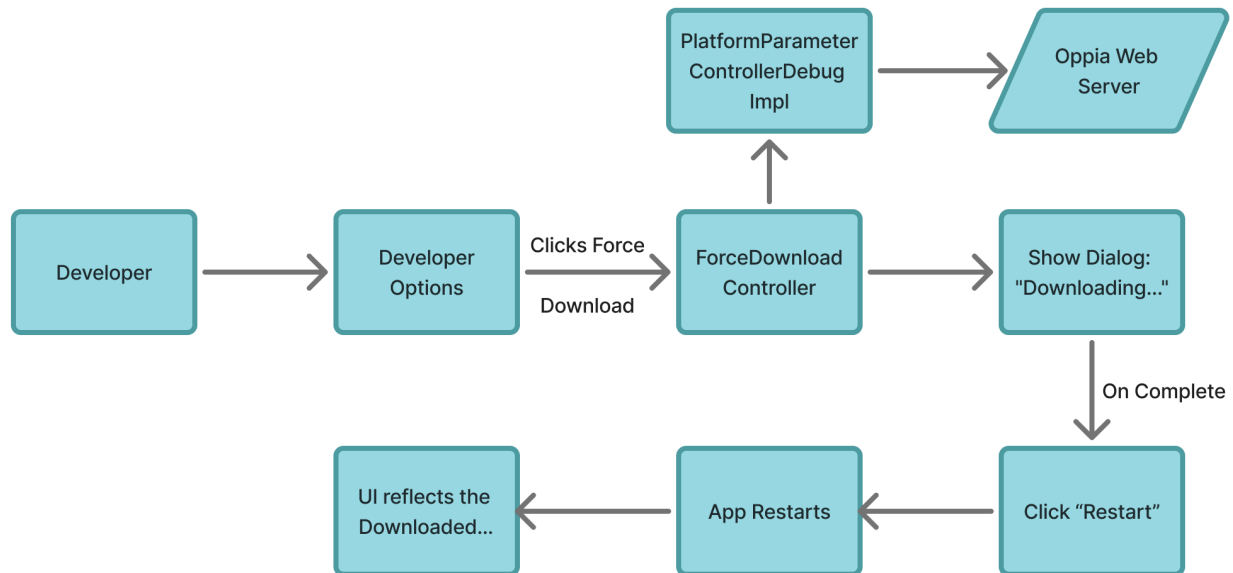
- domain/devoptions/ForceDownloadParametersController.kt
- app/devoptions/ForceDownloadDialogFragment.kt
- app/devoptions/ForceDownloadClickListener.kt
- res/layout/force_download_dialog.xml

Important behavioral adjustments:

- **Fetches values will not be reflected in the UI immediately.** Updates will **only apply after an app restart** when values are initialized from the disk to maintain consistent behavior with the current running app state.
- The UI will trigger the force-download action through a controller call, but the **visible state will remain unchanged** until the user manually restarts the app or it's restarted programmatically.

This will require wiring from UI → Domain Controller → Remote fetch logic.

Flow Diagram:



How remote fetched values are handled Post-Sync and Restart

When the “Force Download Remote Parameters” button is pressed, it triggers a manual fetch of the latest remote platform parameters and feature flags. However, these updates do not immediately reflect in the UI. Instead, the fetched values are stored in a local cache.

- The process begins when the developer clicks “**Force Download Remote Parameters**” from the in-app developer options screen.
- This action explicitly triggers the *PlatformParameterControllerDebugImpl* to initiate a fetch from the **Oppia web server**.
- The controller downloads the **latest values for all platform parameters and feature flags** from the remote server.
- These values are then **written to a persistent remote database** on the device. This database is typically a disk-backed proto storage.
- The values are stored on disk but are **not loaded into the in-memory cache immediately**. This is intentional to preserve app consistency.
- On app restart, the updated values on disk are then loaded into memory via the controller and passed through to the UI.
- Once the app has restarted and loaded the latest values from disk, all UI components, feature toggles, and backend logic that rely on platform parameters will begin reflecting the new remote values.

By using this approach, we ensure that the app's behavior remains consistent with the remote fetch logic, where updates are only reflected after a restart, providing a predictable and stable user experience.

Third-Party Libraries

No new third-party libraries are required.

“Service” Dependencies

The project does not require any Service Dependencies

Key High-Level and Architectural Decisions

Decision 1: How to persist local override values across app restarts?

We have considered the following options:

Alternative 1: Store override values in the same database used for remote sync.

Alternative 2: Introduce a separate persistent proto-based cache:

LocalOverridePlatformParameterDatabase.

Among these, we believe that **Alternative 2 is the best approach**, because:

- **Decoupling:** Separates local overrides from remote data, allowing simpler reset behavior and clarity on override source.
- **Sync Accuracy:** Allows precise conflict resolution: LOCAL_OVERRIDE > REMOTE_SYNC > DEFAULT.
- **UI Alignment:** Makes it easier to visually represent override state (e.g., badges) in the dashboard.
- **Data Integrity:** Avoids accidental loss or corruption of remotely synced data.

The above approaches are contrasted in detail in the following table:

Criteria	Alternative 1: (Shared store)	Alternative 2: (Separate override database)
Data Isolation	Poor – mixing remote + local	Strong separation
Conflict Resolution	Manual and error-prone	Clear priority chain
UI Representation	Requires manual tagging	Can reflect override status cleanly
Reset Behavior	Complicated	Straightforward removal from local db

Decision 2: How to design the new developer dashboard UI for platform parameters and feature flags?

Alternative 1:

Extend the existing Developer Options UI by integrating the list of feature flags and platform parameters directly into the main options RecyclerView.

Alternative 2:

Build the dashboard UI using a dedicated set of Fragments, Presenters, and ViewModels in separate packages (platformparameters and featureflags), with modal dialogs for overrides and restart confirmations.

Alternative 3:

Use a modern declarative approach (e.g., Jetpack Compose) to build the dashboard.

Among these, we believe that **Alternative 2 is the best approach**, because:

- **Modularity & Scalability:**
Building new screens in dedicated packages (e.g., [org/oppia/android/app/devoptions/platformparameters](#) and [.../featureflags](#)) keeps the developer options clean and modular.
- **Integration Simplicity:**
Using standard Android Views avoids introducing new dependencies (as would be needed for Jetpack Compose) and aligns with the existing codebase.

Criteria	Alternative 1 (Extend Existing UI)	Alternative 2 (Dedicated UI Components)	Alternative 3 (Jetpack Compose)
Modularity	Low Modularity - Reuses existing DeveloperOptionsFragment, leading to bloated logic. - UI logic and navigation get entangled with unrelated features. - Makes it harder to manage or reuse individual components.	High Modularity - Each feature (Platform Parameters, Feature Flags) has its own Fragment, Presenter, ViewModel, and layout. - Follows separation of concerns, enabling independent updates and testing.	High Modularity. - Compose encourages modular components (composables), but introducing it in an XML-based codebase requires hybrid architecture and adapter layers.
State Management	- Existing screen shares a single ViewModel or uses Activity-scoped state. - Difficult to persist state across multiple override dialogs or	Robust. - Each screen has a ViewModel scoped to its Fragment.	Compose manages state with remember, mutableStateOf, or StateFlow. - Needs careful scoping to avoid recomposition issues or memory

	navigation stacks.		leaks.
Integration Complexity	- Easy to implement by adding options to existing UI and updating RecyclerView adapters. - No need to refactor navigation or dependencies. - Fastest path for proof-of-concept.	- Requires new navigation entries and DI setup for each screen. - Still aligns with existing architecture and dependency structure (Dagger, XML, MVVM).	- We don't have many established patterns for Jetpack Compose so there are more uncertainties when introducing a new UI. - Existing Dagger bindings and XML layouts may need migration or adapters for compatibility.

Risks and mitigations

Potential Risk	Mitigation
Applying incorrect Parameter Values can lead to unintended app behavior or feature misconfigurations.	Ensure validation of input types and enforce constraints. Implement logging for changes and fallbacks to default values.
Overridden Values Not Persisting After Restart	Ensure overridden values are saved in PersistentCacheStore and correctly retrieved on app launch.
Screen rotation or other config changes (e.g., dark mode) may trigger fragment recreation or destroy transient state like dialogs or unsaved edits.	- Ensure retaining the state after configuration changes by utilizing the <code>`onSavedInstanceState`</code>. - Handle rotation-aware lifecycle events properly. - Add UI tests to verify config-change safety.

Key Objectives for each Milestone

Milestone	Key Objective
Milestone 1	By the end of Milestone 1, the Oppia Android app will include a fully functional Platform Parameter dashboard in the Developer Options that allows developers to inspect Platform Parameters and Feature Flags and a Force Download Remote values functionality. The dashboard will: • Display a complete list of Platform Parameters and Feature Flags in their respective screens along with their current values. • Indicate the sync status (e.g., whether values are default or remotely fetched) using a clear and intuitive visual indicator. • Support downloading of platform parameters from the Oppia Web backend manually at any instant.

	• Show a download progress indicator that persists through configuration changes like screen rotations, ensuring a smooth and non-disruptive user experience. • Be backed by initial unit and UI tests to ensure correctness and visual stability of the new UI. This milestone will lay the groundwork for a reliable and extensible settings panel, serving as a launchpad for further configuration control in Milestone 2.
Milestone 2	At the end of Milestone 2, developers will gain full control over parameter configurations using the Developer Options dashboard. Specifically, they will be able to: • Override and reset the values of Platform Parameters and Feature Flags directly from the UI. • Observe the Restart dialog after making modifications and trying to navigate back. • Verify that the 'Overridden' label appears for the modified flags after restarting. • Ensure that all parameter values are applied reliably by automatically restarting the app upon exiting the platform parameter dashboard. • Maintain pending UI state across device configuration changes (e.g., screen rotation), preventing loss of in-progress modifications. • A complete set of UI and unit tests that verify all critical override and reset functionality. Once complete, this milestone will deliver a proper developer only Dashboard that empowers developers to test and control app behavior through feature flags and platform parameters—fully integrated with Oppia’s architecture and suitable for future scalability.

Implementation Approach

Domain Objects

Dagger Module Injection Changes

Similar to the [HintsAndSolutionDebugModule](#), we'll define separate modules for the debug and production builds.

1. Production Module

Injects the *PlatformParameterControllerProdImpl* into the app during production builds. This module should be used in release builds of the app, where only the remote and default values of platform parameters are relevant.:

```
/** Module for providing production implementation of
 * [PlatformParameterController].
 */
@Module
interface PlatformParameterProdModule {
    @Binds
    fun bindPlatformParameterController(
        impl: PlatformParameterControllerProdImpl
    ): PlatformParameterController
}
```

2. Debug Module

Injects the *PlatformParameterControllerDebugImpl* which extends the production implementation by layering local override behavior. Provides the developer-specific implementation of [PlatformParameterController].

This debug implementation wraps [PlatformParameterControllerProdImpl] and adds support for loading locally overridden platform parameters at app startup. It also provides access to merged platform parameters for the developer options UI. This module should only be included in developer builds.

```
/** Module for providing Debug implementation of [PlatformParameterController]. */
@Module
interface PlatformParameterDebugModule {
    @Binds
    fun bindPlatformParameterController(
        impl: PlatformParameterControllerDebugImpl
    ): PlatformParameterController
}
```

platform_parameter.proto changes:

LocalOverridePlatformParameterDatabase:-

A new proto definition in the existing `platform_parameters.proto` file will be introduced to represent the locally overridden values for platform parameters and feature flags. This database acts independently from the **RemotePlatformParameter Database** to allow the application to reset overridden values back to the synced state without data loss.

This structure defines the local override database which contains lists of platform parameters and feature flags that have been manually overridden by developers or internal testers.

- **overridden_platform_parameter:** A repeated field that stores the list of locally overridden platform parameters.
- **overridden_feature_flag:** A repeated field that stores the list of locally overridden feature flags.

Purpose:

This structure is designed to keep local changes isolated from the remotely synced data. This enables clean restoration of server values and gives a clearer understanding of which values are locally modified.

```
//The database for storing local developer overridden platform parameters and
feature flags.
message LocalOverridePlatformParameterDatabase {
  // Locally overridden platform parameters / feature flags.
  repeated OverriddenPlatformParameter overridden_platform_parameter = 1;

  // A list to store locally overridden feature flags.
  repeated OverriddenFeatureFlag overridden_feature_flag = 2;
}
```

OverriddenPlatformParameter:-

This structure stores a single platform parameter that has been manually overridden locally.

- **id:** Represents the unique *PlatformParameterId* of the overridden parameter. This ID allows the system to identify which parameter has been modified.
- **overridden_value:** Represents the new value that the developer has set for this platform parameter. It uses the *PlatformParameterValue* message to store the type-safe value (boolean, string, or integer).

```
// Corresponds to a developer override of a platform parameter.
message OverriddenPlatformParameter {
  // The unique ID of the overridden platform parameter which corresponds to a
  // PlatformParameterDefintion
  PlatformParameterId id = 1;

  // The overridden value of the platform parameter.
  PlatformParameterValue overridden_value = 2;
}
```

OverriddenFeatureFlag:-

This structure stores a single feature flag that has been manually overridden.

- **id:** Represents the unique *FeatureFlagId* for the overridden feature flag.
- **overridden_is_enabled:** A boolean flag indicating whether the feature has been enabled or disabled by the override.

```
// Corresponds to a developer override of a feature flag.
message OverriddenFeatureFlag {
  // The unique ID of the overridden feature flag.
  FeatureFlagId id = 1;

  // Whether the override enables or disables the feature.
  bool overridden_is_enabled = 2;
}
```

Updated Structure: SyncStatus Enum:

The existing *SyncStatus* enum is updated to include a new status: *LOCAL_OVERRIDE*. This value is essential for representing the status of platform parameters or feature flags that have been manually modified by the user.

- **LOCAL_OVERRIDE:** This new status is used exclusively in the override database. It ensures the system and the UI recognize the value as overridden and not part of the synced remote configuration.

```
// Corresponds to the latest sync status for platform parameters and feature flags.
enum SyncStatus {

  // Indicates that the sync status isn't yet known.
  SYNC_STATUS_UNSPECIFIED = 0;

  // Indicates that the parameter/flag isn't yet synced with the remote server.
  NOT_SYNCED_FROM_SERVER = 1;

  // Indicates the parameter/flag's value has been successfully synced with the
  // remote server
  SYNCED_FROM_SERVER = 2;

  // Indicates the parameter/flag's value have been overridden locally by the
  // developer.
  LOCAL_OVERRIDE = 3;
}
```

User Flows (Controllers and Services)

Here are the implementation details required:

PlatformParameterControllerDebugImpl:-

This is the new implementation of the *PlatformParameterController* specific to the Developer Build.

- It wraps around the real *PlatformParameterControllerProdImpl* ([reference](#)).
- This means it reuses all production behaviour, but with added functionalities layered on top.
- Introduces a new proto-backed persistent store named **localOverrideStore**.
- This store contains a *LocalOverridePlatformParameterDatabase*, which holds developer-defined overrides for PlatformParameters and Feature Flags.
- These overrides are **saved to disk** and **survive app restarts**.
- Core methods like *loadParameters()* and *downloadRemoteParameters()* are delegated to the existing production implementation (prodImpl) to avoid duplication of logic. Additionally, the debug implementation extends this behavior by loading any locally overridden values and applying them on top of the resolved values during app startup, ensuring that the app reflects developer-specified behavior immediately.
- We will also need to add tests for this Debug Implementation of the controller.

```
/**
 * Debug implementation for the controller to manage local developer overridden
 * platform parameters and
 * feature flags.
 */
@Singleton
class PlatformParameterControllerDebugImpl @Inject constructor(
    private val prodImpl: PlatformParameterControllerProdImpl,
    cacheStoreFactory: PersistentCacheStore.Factory
) : PlatformParameterController
```

loadParameters() :

The method *loadParameters()* is responsible for loading platform parameters and feature flags into the app's internal memory (*processState*).

```
/**
 * Initializes and loads the runtime platform parameter and feature flag state.
 * Returns a [DataProvider] that completes once all parameters are loaded.
 */
override fun loadParameters(): DataProvider<Any?>
```

Load Ephemeral Parameters and Flags :

Introducing a new method for computing the list of ephemeral platform parameters from all 3 sources - remote server, local override, default fallback.

This method is introduced in *PlatformParameterControllerDebugImpl* to compute the *effective* set of platform parameters by merging three sources. It ensures that the UI for developer options displays the **correct value currently active** for each parameter, based on this resolution hierarchy.

1. **Local Overrides** – overridden via the Platform parameter dashboard.
2. **Remote Synced Values** – fetched from the Oppia Web Server.
3. **Default Values** – Fallback values defined in the app source code

Execution Steps

1. Load All Supported Parameters

Calls *prodImpl.loadSupportedPlatformParameters()* to fetch the complete list of all platform parameters defined by the app.

2. Fetch Remote Synced Values

Calls *prodImpl.loadRemotePlatformParameters()* to retrieve values fetched from the backend during remote sync.

3. Fetch Local Overrides

Retrieves values from the persistent store *localOverrideStore*, which contains developer-defined local override values saved in *LocalOverridePlatformParameterDatabase*.

4. Merge/Resolution Logic

- For each parameter from the supported list:
 - Check if a **local override** is available. If yes, apply it.
 - Else check if a **remote value** is available. If yes, apply that.
 - Else apply the **default value** defined in the app.

The resolution logic for platform parameters and feature flags will follow the precedence described earlier. To support this behavior in the local override workflow, a new method *loadEphemeralPlatformParameters()* will be introduced in *PlatformParameterControllerDebugImpl*.

Add new proto definition *EphemeralPlatformParameter* and *EphemeralFeatureFlag*

```

// Corresponds to the platform parameter of the current app behavior.
message EphemeralPlatformParameter {
    //The unique ID of this platform parameter which corresponds to a
    PlatformParameterDefinition.
    PlatformParameterId id = 1;

    // The current value of this parameter.
    PlatformParameterValue current_value = 2;

    // Indicates the sync status of the platform parameter
    SyncStatus sync_status = 3;
}

// Corresponds to the feature flag of the current app behavior.
message EphemeralFeatureFlag {
    //The unique ID of this platform parameter which corresponds to a Feature Flag
    Definition.
    FeatureFlagId id = 1;

    // The current value of this Flag.
    bool current_value = 2;

    // Indicates the sync status of the feature flag
    SyncStatus sync_status = 3;
}

```

Platform Parameter:

This method extends the production logic by applying locally overridden values on top of the base implementation. It is used by the Developer Options UI to show the final resolved values along with their origin.

```

/**
 * Returns a merged list of platform parameters by resolving values.
 */
fun loadEphemeralPlatformParameters():
DataProvider<List<EphemeralPlatformParameter>>

```

Feature Flags:

Similar to the above function this will for loading feature Flags.


```
/**
 * Returns a merged list of feature flags by resolving values.
 */
fun loadEphemeralFeatureFlags(): DataProvider<List<FeatureFlag>>
```

How actually will the overriding work?

The `PlatformParameterController` interface in the Oppia Android codebase defines the contract for loading and initializing platform parameters such as feature flags and remote configuration values. Its primary method, `loadParameters()`, is designed to be overridden, allowing for specialized implementations depending on the build type. In the Developer (debug) build, a custom implementation—`PlatformParameterControllerDebugImpl`—extends the production logic with developer-specific overrides.

This debug controller first performs the standard behavior of loading default values (from a textproto configuration file) and fetching remote values from the server. On top of this, it additionally reads from a dedicated `LocalOverridePlatformParameterDatabase`, which stores any developer-defined local overrides set via the Developer Options UI.

The overall parameter resolution process adheres to a clearly defined **priority order: Local Override > Remote > Default**. This means that if a developer overrides a feature flag to `true` in the UI, that value will take precedence, even if the remote server value is `false` or differs from the default.

When such an override occurs, the method `PlatformParameterControllerDebugImpl.updateParameterOverride(paramId, newValue)` is invoked to persist the overridden value in the local override database. During initialization—either on app startup or after fetching new remote values—the controller merges all parameter sources according to the priority rule and emits the combined result to the app. This ensures that developer-specific overrides always take effect, resulting in a consistent and predictable platform parameter state.

Initializing PlatformParameterProcessState

Once the values have been merged, the debug controller initializes the `PlatformParameterProcessState` with the resolved map of parameters. This state object acts as the central in-memory store of the current effective parameter values for the app. By setting `PlatformParameterProcessState` on startup, every part of the app (viewmodels, controllers, etc.) will automatically read from these merged values. In this way, the system guarantees that the correct effective parameter values (including any developer overrides) are used throughout the app. And also the in memory cache does not reflect the changes as soon as a value is overridden rather the changes are reflected after a restart to let the app behaviour remain consistent.

Persistence of Developer Overrides

When a parameter is changed via the Developer Options UI, the new value is written to the *LocalOverridePlatformParameterDatabase*. Importantly, this update does not immediately change the running parameters. In fact, by design updating an override **should not affect the current in-process state**. Instead, the new override is picked up only the next time *loadParameters()* runs (typically after a full app restart). This ensures consistency: the app continues using its existing parameter values until it is restarted, at which point the override takes effect globally.

Injection in rest of the app

The values are injected into the rest of the app using the *@PlatformParameter(id)* or *@FeatureFlag(id)*.

Example:

```
@FeatureFlag(MULTIPLE_CLASSROOMS) private val enableMultipleClassrooms: Boolean
```

Updating Overridden Parameter and Flags:

These methods will allow developers to locally override the value of any platform parameter or feature flags. The new value is written to a persistent store, but the current in-memory platform parameter state (*PlatformParameterProcessState*) is not immediately updated—meaning the change only takes effect on the next app startup.

These methods are intended for development/debug builds to simulate feature flag rollouts or enable experimental features before the server pushes them.

updateOverriddenPlatformParameter():-

```
/**
 * Overrides a platform parameter locally.
 *
 * @param id The identifier of the platform parameter to override.
 * @param newValue The new value for the parameter, represented as a
 * [PlatformParameterValue].
 */
suspend fun updateOverriddenPlatformParameter(id: PlatformParameterId, newValue: PlatformParameterValue)
```

updateOverriddenFeatureFlag():

```
/**
```

```

* Overrides a feature flag locally.
*
* @param id The identifier of the feature flag to override.
* @param isEnabled The new value for the feature flag.
*/
suspend fun updateOverriddenFeatureFlag(id: FeatureFlagId, isEnabled: Boolean)

```

Resetting Overrides:

To revert a developer-specific override and restore the parameter to its remote or default value, use the reset option. So we have methods which **removes a developer-defined override** for a given platform parameter or feature flag from the persistent *localOverrideStore*, effectively resetting it to rely on either:

- Remote sync (if available), or
- The default value defined in the platform parameter definition.

After calling this, the parameter or flag will fall back to its remote or default value. This does not update the current in-process state in [PlatformParameterProcessState] until the next app startup.

resetPlatformParameterOverride():-

```

/**
 * Resets a platform parameter override by removing it from the local override
 * database.
 *
 * @param id The identifier of the platform parameter to reset.
 */
suspend fun resetPlatformParameterOverride(id: PlatformParameterId)

```

resetFeatureFlagOverride():

```

/**
 * Resets a feature flag override by removing it from the local override database.
 *
 * @param id The identifier of the feature flag to reset.
 */
suspend fun resetFeatureFlagOverride(id: FeatureFlagId)

```

Checking if restart is required?

For this purpose we have methods to check whether the restart prompt should be displayed or not. This returns true if a local override exists for the given feature flag or platform parameter but the in-memory PlatformParameterProcessState is still using the old value. Use this in the UI to inform the user that their changes will only apply after restarting the app.

isAppRestartRequiredForParameter():

```
/**
 * Returns whether a restart is required to apply the override for a specific
 * parameter.
 *
 * @param id The identifier of the platform parameter to check.
 * @return a [DataProvider] emitting `true` if restart is needed, `false`
 * otherwise.
 */
fun isAppRestartRequiredForParameter(id: PlatformParameterId):
DataProvider<Boolean>
```

isAppRestartRequiredForFeatureFlag():

```
/**
 * Returns whether a restart is required to apply the override for a specific
 * feature flag.
 *
 * @param id The identifier of the feature flag to check.
 * @return a [DataProvider] emitting `true` if restart is needed, `false`
 * otherwise.
 */
fun isAppRestartRequiredForFeatureFlag(id: FeatureFlagId): DataProvider<Boolean>
```

Force Download Remote Parameters:

The actual logic for downloading remote platform parameters will be handled by the PlatformParameterController, which exposes the download status as an observable stream. In this project, the UI layer is only responsible for observing this status and responding appropriately. The download operation emits values wrapped in **AsyncResult**—starting with **AsyncResult.Loading**, followed by either **AsyncResult.Success** or **AsyncResult.Failure**. These emitted results are then used to trigger the appropriate dialogs or display relevant messages in the UI to inform the user about the outcome of the fetch operation.

There will be a separate controller for fetching the remote values from the server which will call into the *PlatformParameterControllerProdImpl* for the actual downloading and the UI will observe the status received from the controller.

File : domain/devoptions/ForceDownloadParametersController.kt

```
/**
 * Controller class for manually triggering a fetch of remote platform parameters.
 * Intended for developer use only via the Developer Options UI to validate remote
 * sync behavior.
 */
class ForceDownloadParametersController @Inject constructor(
    private val platformParameterController: PlatformParameterController
) {

    /**
     * Initiates a forced download of platform parameters from the remote server.
     *
     * @return a [LiveData] of [SyncStatus] indicating the current sync state
     */
    fun initiateForceDownload(): LiveData<SyncStatus>
}
```

By observing this LiveData we can have the UI represent the loading and completion status.

UI changes:

The UI implementation is pretty straightforward.

Changes in the Developer Options:

- The menu item PlatformParameter and Feature Flags is a simple layout view which we can take from the existing [developer options override app behaviors view.xml](#) where both new menu items will be added which will consist just of TextViews.
- For the Force Downloading Menu Item we need to add a new static TextView with the text “Force Download Remote Parameters” and a “Download” Button in [developer options override app behaviors view.xml](#).
- Introduction of new methods in *DeveloperOptionsOverrideAppBehaviorsViewModel* for routing.

```
/**
 * Routes the user to [PlatformParameterActivity] for managing platform
 * parameters.
 */
```

```

fun onPlatformParameterClicked() {
    platformParameterListener.routeToPlatformParameter()
}
/**
 * Routes the user to [FeatureFlagsActivity] for viewing and modifying
 * feature flags.
 */
fun onFeatureFlagsClicked() {
    featureFlagListener.routeToFeatureFlags()
}

```

Platform Parameter Dashboard:

Platform Parameter Screen:

1. PlatformParameterActivity: It is the activity for holding the Platform Parameter Screen.

```

/** Activity for Platform Parameters. */
class PlatformParameterActivity : InjectableAutoLocalizedAppCompatActivity() {

    @Inject
    lateinit var platformParameterActivityPresenter:
        PlatformParameterActivityPresenter
    ...
}

```

2. PlatformParameterActivityPresenter: It is the presenter for PlatformParameterActivity.

```

/** The presenter for [PlatformParameterActivity]. */
@ActivityScope
class PlatformParameterActivityPresenter @Inject constructor(
    private val activity: AppCompatActivity
)

```

3. PlatformParameterFragment: It is the fragment for the Platform Parameter Screen.

```

/** Fragment to provide functionality to override platform parameters. */
class PlatformParameterFragment : InjectableFragment() {
    @Inject
    lateinit var platformParameterPresenter: PlatformParameterFragmentPresenter
    ...
}

```

4. PlatformParameterFragmentPresenter: It is the presenter for PlatformParameterFragment.

```

/** The presenter for [PlatformParameterFragment]. */
@FragmentScope
class PlatformParameterFragmentPresenter @Inject constructor(
    private val activity: AppCompatActivity,
    private val fragment: Fragment,
    private val platformParameterViewModel: PlatformParameterViewModel,
)

```

5. PlatformParameterViewModel: ViewModel for PlatformParameterFragment.

```

/**
 * [ViewModel] for [PlatformParameterFragment].
 */
@FragmentScope
class PlatformParameterViewModel @Inject constructor(
    private val platformParameterController: PlatformParameterControllerDebugImpl,
) : ObservableViewModel()

```

6. The two layout files will be added *platform_parameter_activity.xml* and *platform_parameter_fragment* :
platform_parameter_activity contains a *FrameLayout* and the *platform_parameter_* will contain *AppBarLayout* with *ToolBar* and a *FrameLayout* with a *RecyclerView* just like [force_network_type_fragment.xml](#).
 For *PlatformParameterFragment* we have to define and create a new view element layout for the *recyclerview* as *platform_parameter_fragment_item_view.xml* which will contain the following elements:
 - *TextView* to display the title of the platform parameter.
 - An input box or a switch depending upon the type of the platform parameter,
 - Another static *TextView* with the text "Sync Status".
 - A view for displaying the current sync status which changes with the type of sync status.
 - Another Container which only shows if the particular platform Parameter is overridden and it contains a *TextView* and a button to reset the overridden platform parameter. On clicking and trying to navigate back then a Dialog appears to restart the app.[\(reference\)](#)

Feature Flag Screen:

1. FeatureFlagActivity: It is the activity for holding the Feature Flag Screen.

```

/** Activity for Feature Flags. */
class FeatureFlagActivity : InjectableAutoLocalizedAppCompatActivity() {

    @Inject
    lateinit var featureFlagActivityPresenter:
        FeatureFlagActivityPresenter

    ...
}

```

2. FeatureFlagActivityPresenter: It is the presenter for FeatureFlagActivity.

```
/** The presenter for [FeatureFlagActivity]. */
@ActivityScope
class FeatureFlagActivityPresenter @Inject constructor(
    private val activity: AppCompatActivity
)
```

3. FeatureFlagFragment: It is the fragment for the Feature Flag Screen.

```
/** Fragment to provide functionality to override platform parameters. */
class FeatureFlagFragment : InjectableFragment() {
    @Inject
    lateinit var featureFlagPresenter: FeatureFlagFragmentPresenter
    ...
}
```

4. FeatureFlagFragmentPresenter: It is the presenter for FeatureFlagFragment.

```
/** The presenter for [FeatureFlagFragment]. */
@FragmentScope
class FeatureFlagFragmentPresenter @Inject constructor(
    private val activity: AppCompatActivity,
    private val fragment: Fragment,
    private val featureFlagViewModel: FeatureFlagViewModel,
)
```

5. FeatureFlagViewModel: ViewModel for FeatureFlagFragment.

```
/**
 * [ViewModel] for [FeatureFlagFragment].
 */
@FragmentScope
class FeatureFlagViewModel @Inject constructor(
    private val platformParameterController: PlatformParameterControllerDebugImpl,
) : ObservableViewModel()
```

6. The two layout files will be added *feature_flag_activity.xml* and *feature_flag_fragment* :
feature_flag_activity contains a *FrameLayout* and the
feature_flag_fragment will contain *AppBarLayout* with *ToolBar* and a
FrameLayout with a *RecyclerView* just like [force_network_type_fragment.xml](#).
For FeatureFlagFragment we have to define and create a new view element layout for
the recyclerview as *feature_flag_fragment_item_view.xml* which will contain the
following elements:

- *TextView* to display the title of the platform parameter.
- Switch for enable/disable that feature.
- Another static *TextView* with the text “Sync Status”.
- A view for displaying the current sync status which changes with the type of sync status.
- Another Container which only shows if the particular feature flag is overridden and it contains a *TextView* and a button to reset the overridden feature flag. On clicking and trying to navigate back then a Dialog appears to restart the app.([reference](#))

Restart Dialog:

For the Restart Dialog similar approach will be followed. The list of files include:

1. *fragment_confirm_restart_dialog.xml* : Will contain a *TextView* and a button.
2. *ConfirmRestartDialogFragment.kt*

```
/** DialogFragment that is used to restart the app. */
class ConfirmRestartDialogFragment : InjectableDialogFragment() {

    @Inject

    lateinit var confirmRestartDialogFragmentPresenter:
    ConfirmRestartDialogFragmentPresenter

    ...
}
```

3. *ConfirmRestartDialogFragmentPresenter.kt*

```
/** The presenter for [ConfirmRestartDialogFragment]. */
@FragmentScope
class ConfirmRestartDialogFragmentPresenter @Inject constructor(
    private val fragment: Fragment,
    private val activity: AppCompatActivity
)
```

This dialog(which is non-cancelable) is shown when the user overrides a platform parameter or feature flag and must restart the app to apply the changes. It provides one button **Restart**, which restarts the app. The layout used for this dialog is defined in *R.layout.fragment_confirm_restart_dialog*.

Force Downloading:

This will need the following files:

1. *ForceDownloadDialogFragment.kt*: This Dialog will serve as an interface for downloading remote parameters. It shows “Downloading” when the parameters are being downloaded and when the download is complete then the “Restart” button becomes active.

```

/**
 * DialogFragment that is used to manually trigger a force download of platform
 parameters.
 */
class ForceDownloadDialogFragment : InjectableDialogFragment() {

    @Inject
    lateinit var forceDownloadDialogFragmentPresenter:
ForceDownloadDialogFragmentPresenter

    ...
}

```

2. ForceDownloadDialogFragmentPresenter.kt:

```

/**
 * The presenter for [ForceDownloadDialogFragment].
 */
@FragmentScope
class ForceDownloadDialogFragmentPresenter @Inject constructor(
    private val fragment: Fragment,
    private val activity: AppCompatActivity,
    private val platformParameterController: PlatformParameterControllerDebugImpl
)

```

3. force_downloading_dialog_fragment.xml :

This is the layout file which will contain the title textview “Remote Platform Parameters” and the dynamic description textView which will show “Downloading ” and “ Download Completed Successfully ” and two buttons “Cancel ” and “Restart” .

DATA FLOW FOR USER INTERACTIONS:

1. Data Source

a. RemotePlatformParameterAndFeatureFlagDatabase (Proto)

- This proto file holds the **canonical set of remote values** fetched from the backend or bundled with the app.
- Used during initialization and sync via *PlatformParameterController*.

b. LocalOverridePlatformParameterDatabase (New Proto)

- This is the **newly introduced proto** storing locally overridden values.
- Ensures overridden values persist across app restarts and are prioritized over remote values when present.

2. Controller Layer

a. PlatformParameterController/PlatformParameterControllerDebugImpl

- Responsible for:
 - Fetching values from local proto stores.
 - Combining local, remote and overridden values to yield effective parameter values.
 - Managing logic for syncing and updating overrides

b. PlatformParameterController/PlatformParameterControllerProdImpl

- Responsible for:
 - Downloading values from the Oppia Web Server(*downloadRemoteParameters()*).
 - Loading the values at the app startup.(by *loadParameters()*)

3. ViewModel Layer

a. PlatformParametersViewModel / FeatureFlagsViewModel

- Acts as a bridge between the controller and the UI.
- Subscribes to LiveData or Flow from controllers.
- Transforms raw data into RecyclerView item view models to represent each parameter or flag.
- Also handles UI logic like showing modals or triggering restart confirmation when overrides are changed.

4. UI Layer

a. Fragment & Presenter

- Fragments (*PlatformParametersFragment*, *FeatureFlagsFragment*) inflate the layout and initialize the RecyclerView.
- Presenters create the actual adapter (*BindableAdapter*) and bind view models to their corresponding item views.
- UI includes:
 - List of parameters/flags with current value along with Sync Status.
 - Override switch or input field.
 - Optional “Restart Required” confirmation dialog when override is updated.

5. User Interaction Flow

Example: Overriding a Platform Parameter

1. **User taps a platform parameter toggle or changes its value.**
→ The Presenter calls `ViewModel.onOverrideToggled(newValue)` (or an equivalent method for edit).

2. **ViewModel delegates to PlatformParameterControllerDebugImpl.**
→ Calls `overridePlatformParameter(parameterName, newValue)`.
3. **In PlatformParameterControllerDebugImpl:**
 - It compares `newValue` with the current **effective parameter value** (i.e., the value that would currently be returned to the app).
 - It also considers any **existing local override** when making this decision.
 - **Case A – Value is different (actual override):**
→ The override is stored in the `LocalOverridePlatformParameterDatabase`.
→ `restartRequired = true` is returned.
 - **Case B – Value is same (no effective change):**
→ No override is stored or modified.
→ `restartRequired = false` is returned.
 - **Case C – Value reverts to remote/default (removing an override):**
→ The existing local override is removed.
→ If this causes the effective value to change,
→ `restartRequired = true` is returned.
 - **Case D – Revert or change leads to same effective value (redundant):**
→ No meaningful change; override not updated.
→ `restartRequired = false`.
 - **Case E – Multiple changes (batch override):**
→ Controller evaluates **net effect** across all parameter overrides.
→ If any change causes a difference in the effective state,
→ `restartRequired = true, else false`.
4. **ViewModel receives the restartRequired flag.**
→ Updates a `LiveData<Boolean>` to indicate whether an app restart is needed.
5. **Presenter observes this LiveData.**
→ If true, it shows `ConfirmRestartDialogFragment`.
6. **User confirms restart.**
→ The app is restarted, and updated parameters take effect.

Test data changes

NA

Script & CI changes

The project does not require any Script & CI changes.

Documentation changes

The project does not require any Documentation changes.

Implementation Plan

Milestone Table

Milestone 1 (Jun 2 – July 4)

Key objective for this milestone:

Display a list of platform parameters and feature flags from the Developer Options menu, along with their current values and sync statuses. Support force downloading of remote values. The indicator for downloading state should survive device rotations.

No.	Description of PR / action		Prerequisite steps	Target date	
				PR creation	PR merge
1.1	Introduce the model protos required for the project.(eg: <i>LocalOverridePlatformParameterDatabase</i>)		None	June 3	June 4
1.2	Introduce the <i>PlatformParameterControllerDebugImpl</i> and module to inject it.		1.1	June 6	June 12
1.3	Introduce menu item, listener, controller, dialog fragment for the Force Downloading.		None	June 10	June 14
1.4	Feature Flag Screen				
	1.4.1	Introduce menu item and listener for Feature Flag.	None	June 11	June 12
	1.4.2	Introduce the UI elements and kotlin files required for Feature Flags.	1.1, 1.2, 1.4.1	June 14	June 18
	1.4.3	Write tests for the feature Flags.	1.4.1, 1.4.2	June 18	June 21
PM Demo	Initial dashboard with flag list & correct values along with force downloading.		1.1, 1.2, 1.3, 1.4	June 24	
1.5	Platform Parameter Screen				
	1.5.1	Introduce menu item and listener for Platform Parameter.	None	June 21	June 22
	1.5.2	Introduce the UI elements and kotlin files required for Platform Parameters.	1.1, 1.2, 1.5.1	June 24	June 26

	1.5.3	Write tests for the Platform Parameters.	1.5.1,1.5.2	June 26	June 28
1.6	Implement any suggestion or comments from the PM and any remaining task..		PM Demo	July 1	July 3

Milestone 2 (July 26 – August 27)

Key objective for this milestone:

Support overwriting and resetting both feature flags and platform parameters back to their original remote server / default fallback values, with persistence across rotation and a forced app restart to apply changes.

No.	Description of PR / action		Prerequisite steps	Target date	
				PR creation	PR merge
2.1	Overrides for Feature Flags				
	2.1.1	Making proto changes for feature flag overrides.	M1	July 26	July 28
	2.1.2	Making controller changes for feature flag overrides	2.1.1	July 28	Aug 3
	2.1.3	UI and domain flow for feature flag overrides and implementing configuration change survival.	2.1.2	Aug 4	Aug 7
	2.1.4	Testing for feature flag overrides (unit tests, integration tests, edge cases).	2.1.3	Aug 8	Aug 12
2.2	Overrides for Platform Parameters.				
	2.2.1	Making proto changes for platform parameter overrides	None	Aug 9	Aug 11
	2.2.2	Making controller changes for platform parameter overrides and implementing configuration change survival.	2.2.1	Aug 12	Aug 16
	2.2.3	UI and domain flow for platform parameter overrides	2.2.2	Aug 17	Aug 20
	2.2.4	Testing for platform parameters overrides (unit tests, integration tests, edge cases)	2.2.3	Aug 20	Aug 22

PM Demo	Show overriding tasks and other completed works.	2.4	Aug 21	
2.3	Reset Functionality for Both Feature Flags and Platform Parameters	2.2	Aug 22	Aug 24
2.4	Adding working "Restart" dialogs for overrides and reset implementation.	2.3	Aug 24	Aug 25
2.5	Implement any suggestion or comments from PM.		Aug 26	Aug 28

Rest other days which are not included in the table will be used as a backup to follow up any pending tasks.

Feature Flags

No additions or modifications to existing feature flags are required.

Future Work

Real-time Parameter Propagation

Currently, overridden platform parameters only take effect after restarting the app. In the future, support for hot-swapping values at runtime (for safe types like booleans or strings) could be introduced. This would avoid the need for restarts during rapid testing workflows.